

nanopal.py — детальный разбор кода

Код-фёрст агент на Python с песочницей и OpenRouter

Документация проекта nanopal

July 23, 2026

Contents

1	Введение	2
2	Shebang и PEP 723 inline metadata	2
3	Docstring	2
4	Импорты	2
5	Конфигурация	3
5.1	Загрузка задачи	3
5.2	Загрузка .env	3
5.3	Константы	3
6	System Prompt	3
7	Безопасные builtins	3
8	Вспомогательные функции	4
8.1	clip()	4
8.2	trim_messages()	4
8.3	safe_path()	4
9	Инструменты	4
9.1	list_dir()	4
9.2	read_file()	5
9.3	write_file()	5
9.4	exec_cmd()	5
10	final_answer()	6
11	extract_python()	6
12	Функции вывода (Rich)	6
13	main() — главный цикл	6
13.1	Обработка ошибок	7
13.2	Retry с backoff	7
14	Заключение	7

1 Введение

nanopal.py — минимальный код-фёрст агент на Python (~280 строк), построенный на базе Nano Harness из контекстного курса Hugging Face (Unit 6).

Агент получает задачу на естественном языке, вызывает LLM через OpenRouter, модель генерирует исполняемый Python-код, который запускается в изолированном окружении с ограниченным набором инструментов. Результат возвращается модели — цикл повторяется до 50 шагов, пока задача не будет решена.

Данный документ содержит построчный разбор каждого логического блока файла `nanopal.py`.

2 Shebang и PEP 723 inline metadata

```
#!/usr/bin/env -S uv run --script
# /// script
# dependencies = [
#     "openai>=1.0.0",
#     "rich>=13.0.0",
# ]
# ///
```

Shebang — при запуске через `symlink`, `uv run --script` автоматически создаёт виртуальное окружение, устанавливает зависимости и запускает скрипт. **PEP 723** позволяет запускать `nanopal` без предварительной установки пакетов.

3 Docstring

```
□ WARNING: This is a LEARNING TOOL, NOT a security boundary.
    The exec() sandbox (filtered builtins) can be bypassed via Python's
    __class__.__bases__.__subclasses__() chain.
```

Дисклеймер о безопасности. Фильтрация `builtins` в `exec()` не является настоящей песочницей. Через `__class__.__bases__.__subclasses__()` можно добраться до классов из уже загруженных модулей — настоящая изоляция требует отдельного OS-процесса или контейнера.

4 Импорты

```
import io                # StringIO для exec()
import os                # переменные окружения
import re                # извлечение кода из ответа модели
import subprocess        # запуск shell-команд
import sys               # argv[1], exit()
import time              # sleep() для backoff
from contextlib import redirect_stderr, redirect_stdout # перехват вывода exec()
from pathlib import Path  # resolve(), relative_to()

from openai import OpenAI # клиент OpenRouter API
from rich.console import Console # форматирование вывода
from rich.panel import Panel
from rich.rule import Rule
```

```
from rich.syntax import Syntax
```

5 Конфигурация

5.1 Загрузка задачи

```
TASK = sys.argv[1] if len(sys.argv) > 1 else ""
```

Первый аргумент командной строки — задача для агента.

5.2 Загрузка .env

```
_env_path = Path(__file__).resolve().parent / ".env"
```

Примитивный парсер .env без внешних зависимостей. Path(__file__).resolve() раскрывает symlink — иначе скрипт искал бы .env в ~/.local/bin/.

5.3 Константы

Константа	Значение	Описание
MODEL	deepseek/deepseek-v4-flash	Модель через OpenRouter
WORKSPACE	cwd	Корневая директория
MAX_STEPS	50	Максимум итераций
TEMPERATURE	0.2	Детерминированный вывод
TIMEOUT_S	30	Таймаут subprocess
MAX_CHARS	8000	Лимит вывода
ALLOW_WRITE	False	Запись отключена
ALLOW_COMMANDS	ls, cat, pwd, echo, head, tail, wc, rg	Белый список

6 System Prompt

Задаёт модель поведения агента:

1. **Code-first** — только Python-код, без prose
 2. **Инструменты** — список функций с описанием
 3. **Termination** — final_answer() для завершения
 4. **Ограничения** — workspace, allowlist, лимит вывода
 5. **import** — явно указано, что недоступен
 6. **Подсказка** — исключать .venv/, __pycache__/, node_modules/
-

7 Безопасные builtins

Белый список builtins для exec(). Исключены:

- __import__ — модель не должна импортировать модули
- open — файлы только через инструменты

- `exec`, `eval`, `compile` — предотвращает код внутри кода
 - `input` — интерактив не нужен
 - `SystemExit`, `KeyboardInterrupt` — наследуются от `BaseException`, не ловятся `except Exception`
-

8 Вспомогательные функции

8.1 `clip()`

```
def clip(x, n=MAX_CHARS):
    s = str(x)
    return s[:n] + "\n...[truncated]" if len(s) > n else s
```

Обрезает строку до n символов.

8.2 `trim_messages()`

```
MAX_HISTORY = 20
```

```
def trim_messages(messages, keep=MAX_HISTORY):
    if len(messages) <= keep + 2:
        return messages
    return messages[:2] + messages[-(keep):]
```

Предотвращает переполнение контекста. Сохраняет `system + task` (индексы 0 и 1) и последние `keep` сообщений.

8.3 `safe_path()`

```
def safe_path(user_input):
    ws = Path(WORKSPACE).resolve()
    requested = (ws / user_input).resolve()
    try:
        requested.relative_to(ws)
    except ValueError:
        raise ValueError(f"Path '{user_input}' escapes workspace")
    return requested
```

Главная защита от directory traversal. Раскрывает symlink через `.resolve()`, проверяет через `.relative_to()`.

9 Инструменты

9.1 `list_dir()`

```
def list_dir(path="."):
    p = safe_path(path)
    if not p.is_dir():
        raise NotADirectoryError(str(p))
    return sorted(x.name + ("/" if x.is_dir() else "") for x in p.iterdir())
```

Безопасный листинг. Возвращает только имена, директории с `/`.

9.2 read_file()

```
def read_file(path, max_chars=4000):
    p = safe_path(path)
    content = p.read_text(encoding="utf-8", errors="replace")
    return clip(content, min(max_chars, MAX_CHARS))
```

Чтение с двумя уровнями лимита. errors="replace" для бинарных файлов.

9.3 write_file()

```
def write_file(path, content):
    if not ALLOW_WRITE:
        raise PermissionError("write_file is disabled")
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(str(content), encoding="utf-8")
```

Единственный способ мутации ФС. Отключена по умолчанию.

9.4 exec_cmd()

```
def exec_cmd(args):
    # 1. Проверка имени
    if args[0] not in ALLOW_COMMANDS:
        raise PermissionError(...)

    # 2. Блокировка rg --pre и --pre=
    if args[0] == "rg":
        for arg in args:
            if arg == "--pre" or arg.startswith("--pre="):
                raise PermissionError(...)

    # 3. Path confinement аргументов
    checked_args = [args[0]]
    for arg in args[1:]:
        if arg.startswith("-"):
            checked_args.append(arg)
            continue
        if arg.startswith("/") or "/.." in arg or arg in (".", ".."):
            try:
                checked_args.append(str(safe_path(arg)))
            except ValueError:
                raise PermissionError(...)
        else:
            checked_args.append(arg)

    # 4. Запуск с изоляцией
    result = subprocess.run(
        checked_args,
        capture_output=True,
        timeout=TIMEOUT_S,
        text=True,
        cwd=WORKSPACE,
        env={"PATH": os.environ.get("PATH", "")},
    )
```

Четыре уровня защиты:

1. **Allowlist** — только `ls`, `cat`, `pwd`, `echo`, `head`, `tail`, `wc`, `rg`
 2. **Блокировка `rg --pre`** — ловит и `--pre`, и `--pre=sh`
 3. **Path confinement** — пути за пределами `workspace` блокируются
 4. **Изоляция** — `cwd=WORKSPACE`, `env` без секретов
-

10 final_answer()

```
def final_answer(value):
    global DONE, FINAL_RESULT
    DONE = True
    FINAL_RESULT = value
    return value
```

Сигнал завершения. После вызова цикл прекращается.

11 extract_python()

```
def extract_python(content):
    blocks = re.findall(r"```(?:python|py)?\s*\n?(.*?)```", content, re.DOTALL)
    if blocks:
        return "\n".join(b.strip() for b in blocks).strip()
    return content.strip()
```

Извлекает Python-код из ответа модели. Поддерживает ````python`, ````py`, `````. Возвращает все блоки, склеенные через `\n`.

12 Функции вывода (Rich)

```
def print_header():          # Panel с моделью, workspace, задачей
def print_step_rule(step):   # Rule-разделитель шага
def print_model_output():    # Panel + Syntax с подсветкой Python (Monokai)
def print_observation():     # Panel с результатом (зелёный/красный)
def print_final():           # Panel-fit с финальным ответом
```

Весь вывод переведён на Rich. Спиннер `console.status()` показывает “Thinking...” с анимацией во время ретраев.

13 main() — главный цикл

1. Инициализация: `OpenAI client`, `messages = [system, task]`
2. Цикл до `MAX_STEPS`:
 - a. Вызов LLM с `retry` (3 попытки, `exponential backoff`, спиннер)
 - b. Извлечение кода через `extract_python()`
 - c. `exec()` с `_SAFE_BUILTINS` + инструменты
 - d. Перехват `stdout/stderr` через `redirect_stdout/stderr`

```

    e. Обработка ошибок (FileNotFoundError, PermissionError, Timeout, ...)
    f. Если DONE — print_final(), break
    g. Иначе — print_observation(), добавить в history
    h. trim_messages() — обрезка истории
3. Если цикл завершился без DONE — "Max steps reached"

```

13.1 Обработка ошибок

Исключение	Результат
FileNotFoundError	Файл не найден
PermissionError	Действие запрещено
subprocess.TimeoutExpired	Команда зависла
Exception (общий)	Error: тип: сообщение

Все ошибки возвращаются модели как observations — она адаптируется.

13.2 Retry с backoff

```

for attempt in range(3):
    try:
        response = client.chat.completions.create(...)
        succeeded = True
        break
    except Exception as e:
        if attempt == 2: break
        wait = 2 ** attempt
        time.sleep(wait)

```

3 попытки с exponential backoff (1с, 2с, 4с). Спиннер console.status() показывает статус во время ожидания.

14 Заключение

nanopal.py — учебный код-фёрст агент, демонстрирующий:

- Цикл агента (LLM → код → exec → наблюдение → повтор)
- Песочницу на уровне инструментов (safe_path, allowlist, env isolation)
- Обработку ошибок как часть цикла (ошибка → observation)
- Управление контекстом (trim_messages)
- Отказоустойчивость (retry с backoff)

Не является production-решением — exec() в том же процессе не даёт настоящей изоляции.